Specyfikacja systemu GPUShare

Finalna dokumentacja projektowa

Konrad Burdach

Hubert Błonowski

Natan Dworok

Mateusz Giermek

Wojciech Królikowski

1 lutego 2026

Streszczenie. GPUShare jest platformą do udostępniania mocy obliczeniowej kart graficznych pomiędzy właścicielami sprzętu i klientami. System dostarcza katalog GPU, proces wynajmu z rozliczeniem oraz telemetrię, a także mechanizmy weryfikacji i obsługi sporów.

Spis treści

1 Cel i ogólna koncepcja systemu **3**

2 Zakres i role użytkowników **3**
2.1 Role . . . 3
2.2 Założenia i ograniczenia . . . 3

3 Funkcjonalności systemu **3**
3.1 Klient . . . 3
3.2 Właściciel GPU . . . 3
3.3 Administrator . . . 4

4 Architektura systemu (komponenty) **4**

5 Kluczowe obiekty biznesowe **4**
5.1 Diagram klas . . . 6

6 Komunikacja w systemie **7**
6.1 Założenia . . . 7
6.2 Endpointy API . . . 7
6.3 Struktury komunikatów (JSON) . . . 7
6.3.1 Rejestracja urządzenia . . . 7
6.3.2 Status urządzenia . . . 8
6.3.3 Wynajem urządzenia . . . 8
6.4 Sekwencje wymiany wiadomości . . . 9
6.4.1 Sekwencja: Rejestracja GPU . . . 9
6.4.2 Sekwencja: Wynajem GPU . . . 10

7 Przypadki użycia / historyjki i kryteria akceptacji **11**
7.1 Diagram przypadków użycia . . . 11
7.2 Historyjki użytkowników . . . 12
7.3 Rejestracja GPU . . . 13
7.4 Wynajem GPU . . . 13
7.5 Zgłoszenie sporu . . . 13

8 Diagramy stanów **13**
8.1 Diagram stanów GPU . . . 13
8.2 Diagram stanów wypożyczenia . . . 14
8.3 Diagram stanów użytkownika . . . 16
8.4 Diagram stanów sporu . . . 16

9 Diagramy aktywności **17**
9.1 Diagram aktywności wypożyczenia . . . 17
9.2 Diagram aktywności udostępniania GPU . . . 19
9.3 Diagram aktywności sporu . . . 21

1 Cel i ogólna koncepcja systemu

System GPUShare jest platformą pośredniczącą w udostępnianiu mocy obliczeniowej kart graficznych pomiędzy właścicielami sprzętu a użytkownikami końcowymi. Celem systemu jest:

- umożliwienie właścicielom GPU monetyzacji posiadanego sprzętu,
- umożliwienie klientom wynajmu GPU bez konieczności zakupu sprzętu,
- zapewnienie administratorom narzędzi do weryfikacji, rozliczalności i obsługi sporów.

System realizuje powyższe cele przez architekturę mikroserwisową, centralne API oraz agenta uruchamianego na maszynie udostępniającej GPU.

2 Zakres i role użytkowników

2.1 Role

- **Klient** – wyszukuje GPU, składa zamówienia, monitoruje wynajem, zgłasza niezgodności i wystawia opinie.
- **Właściciel GPU** – rejestruje urządzenia, ustawia dostępność, monitoruje obciążenie i otrzymuje płatności.
- **Administrator** – weryfikuje konta, rozstrzyga spory, monitoruje statystyki.

2.2 Założenia i ograniczenia

- Komunikacja aplikacji klienckiej z backendem odbywa się poprzez HTTPS/JSON.
- Autoryzacja jest realizowana tokenami JWT.
- Telemetria czasu rzeczywistego realizowana jest przez WebSocket/SSE.

3 Funkcjonalności systemu

3.1 Klient

- wyszukiwanie dostępnych kart graficznych według parametrów (model, VRAM, cena, lokalizacja),
- wynajem wybranego GPU na określony czas,
- śledzenie statusu aktywnego wypożyczenia (czas, koszty, podstawowe metryki),
- zgłoszenie niezgodności sprzętu z ofertą,
- wystawienie opinii po zakończeniu wynajmu.

3.2 Właściciel GPU

- dodawanie GPU do systemu (rejestracja urządzenia),
- zarządzanie dostępnością (włącz/wyłącz udostępnianie),
- monitorowanie obciążenia urządzenia,
- automatyczne rozliczanie i płatności po zakończeniu wypożyczenia.

3.3 Administrator

- weryfikacja kont użytkowników i właścicieli GPU,
- rozwiązywanie sporów,
- monitoring statystyk użycia.

4 Architektura systemu (komponenty)

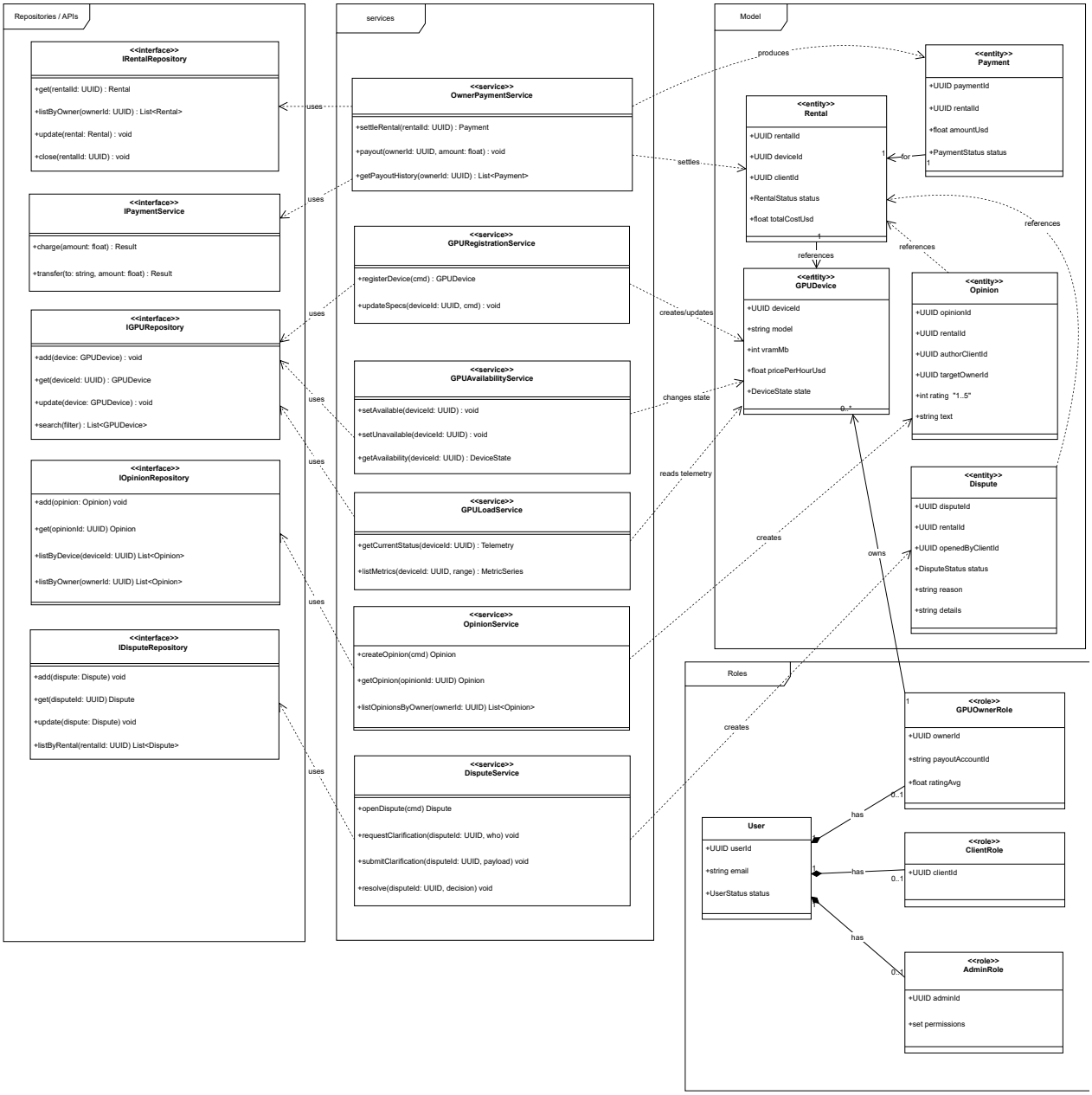
System jest oparty o architekturę mikroserwisową. Komponenty:

- **Client App (Frontend)** – aplikacja webowa użytkownika.
- **Marketplace Service** – katalog GPU, wyszukiwanie, parametry ofert.
- **Order/Billing Service** – rezerwacje, wynajem, rozliczenia.
- **Node Agent** – agent na maszynie właściciela GPU: heartbeat, status, uruchamianie środowiska obliczeniowego.

5 Kluczowe obiekty biznesowe

Obiekt	Opis
User	Użytkownik systemu; posiada role (Klient, Właściciel GPU, Administrator).
GPUDevice	Urządzenie GPU z parametrami (model, VRAM, cena, lokalizacja), stanem i właścicielem.
Rental	Proces wynajmu GPU wraz ze statusem, czasem trwania i kosztami.
Payment	Rozliczenie za wypożyczenie (saldo, blokady środków, finalizacja).
Dispute	Spór zgłoszony przez klienta dotyczący niezgodności/awarii/rozliczeń.
Opinion	Ocena po zakończeniu wypożyczenia.

5.1 Diagram klas



Rysunek 1: UML class diagram

6 Komunikacja w systemie

6.1 Założenia

- Podstawowa komunikacja: **REST API (HTTP/HTTPS)** + **JSON**.
- Autoryzacja: **JWT** przesyłany w nagłówku `Authorization: Bearer <token>`.
- Telemetria czasu rzeczywistego: **WebSocket** lub **Server-Sent Events (SSE)**.

6.2 Endpointy API

Metoda	Endpoint	Opis
POST	/api/devices	Rejestracja urządzenia GPU (właściciel / agent).
GET	/api/devices/{id}	Pobranie tożsamości/konfiguracji urządzenia.
GET	/api/devices/{id}/status	Aktualny stan urządzenia (availability + metryki).
GET	/api/devices	Lista/wyszukiwanie urządzeń (parametry, filtry).
POST	/api/orders	Inicjalizacja wynajmu.
GET	/api/orders/{id}	Status wypożyczenia.
POST	/api/disputes	Zgłoszenie sporu.
GET	/api/disputes/{id}	Status sporu.

6.3 Struktury komunikatów (JSON)

6.3.1 Rejestracja urządzenia

Request: `POST /api/devices`
Nagłówki: `Content-Type: application/json`

```
{
  "name": "Moja karta RTX 4090",
  "gpu_model": "NVIDIA GeForce RTX 4090",
  "vram_mb": 24576,
  "cuda_cores": 16384,
  "price_per_hour_usd": 0.45,
  "driver_version": "535.104",
  "supported_frameworks": ["pytorch", "tensorflow"]
}
```

Response: `201 Created`

```
{
  "device_id": "550e8400-e29b-41d4-a716-446655440000",
  "owner_id": "user_12345",
  "status": "AVAILABLE",
  "created_at": "2026-01-06T12:34:56Z"
```

```
}
```

6.3.2 Status urządzenia

Response: `GET /api/devices/{device_id}/status`

```
{
  "deviceId": "123",
  "state": "AVAILABLE",
  "temperatureC": 68,
  "utilizationPercent": 72,
  "memoryUsedMb": 10240,
  "lastHeartbeat": "2026-01-06T12:34:56Z"
}
```

6.3.3 Wynajem urządzenia

Request: `POST /api/orders`

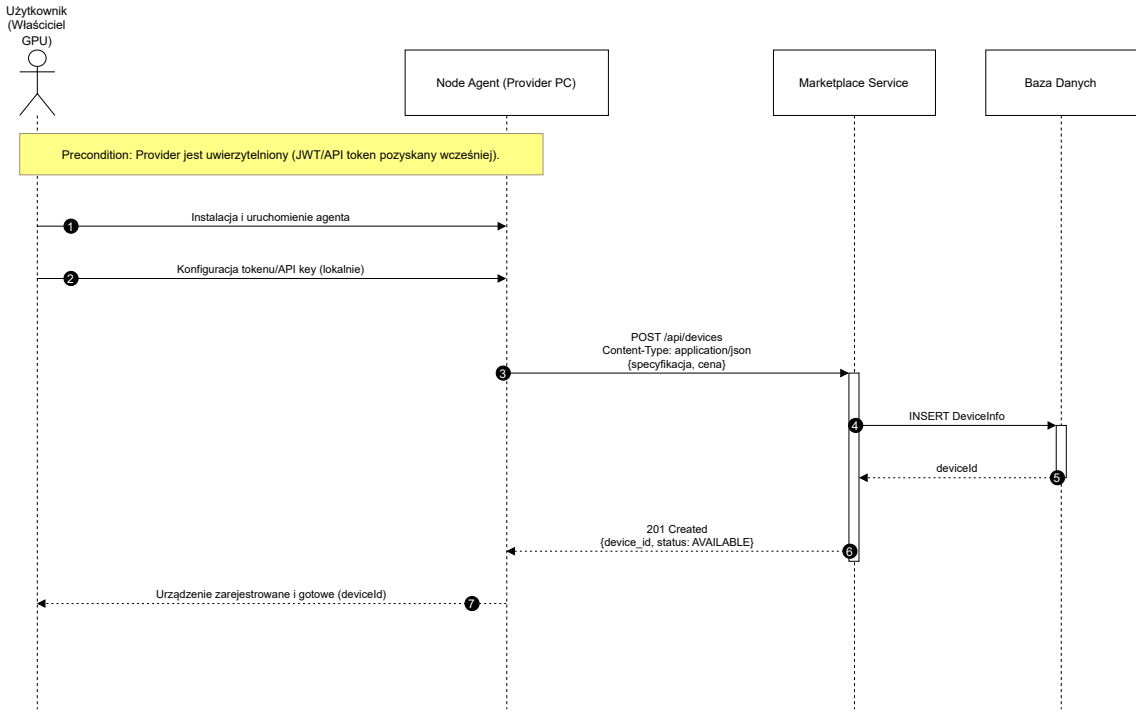
```
{
  "device_id": "550e8400-e29b-41d4-a716-446655440000",
  "docker_image": "pytorch/pytorch:2.0-cuda11.7",
  "duration_hours": 2
}
```

Response: `201 Created`

```
{
  "order_id": "ord_123456789",
  "status": "WAITING_FOR_START",
  "connection_details": {
    "host": "node-01.gpushare.net",
    "port": 443,
    "protocol": "wss"
  },
  "total_reserved_cost": 0.90
}
```

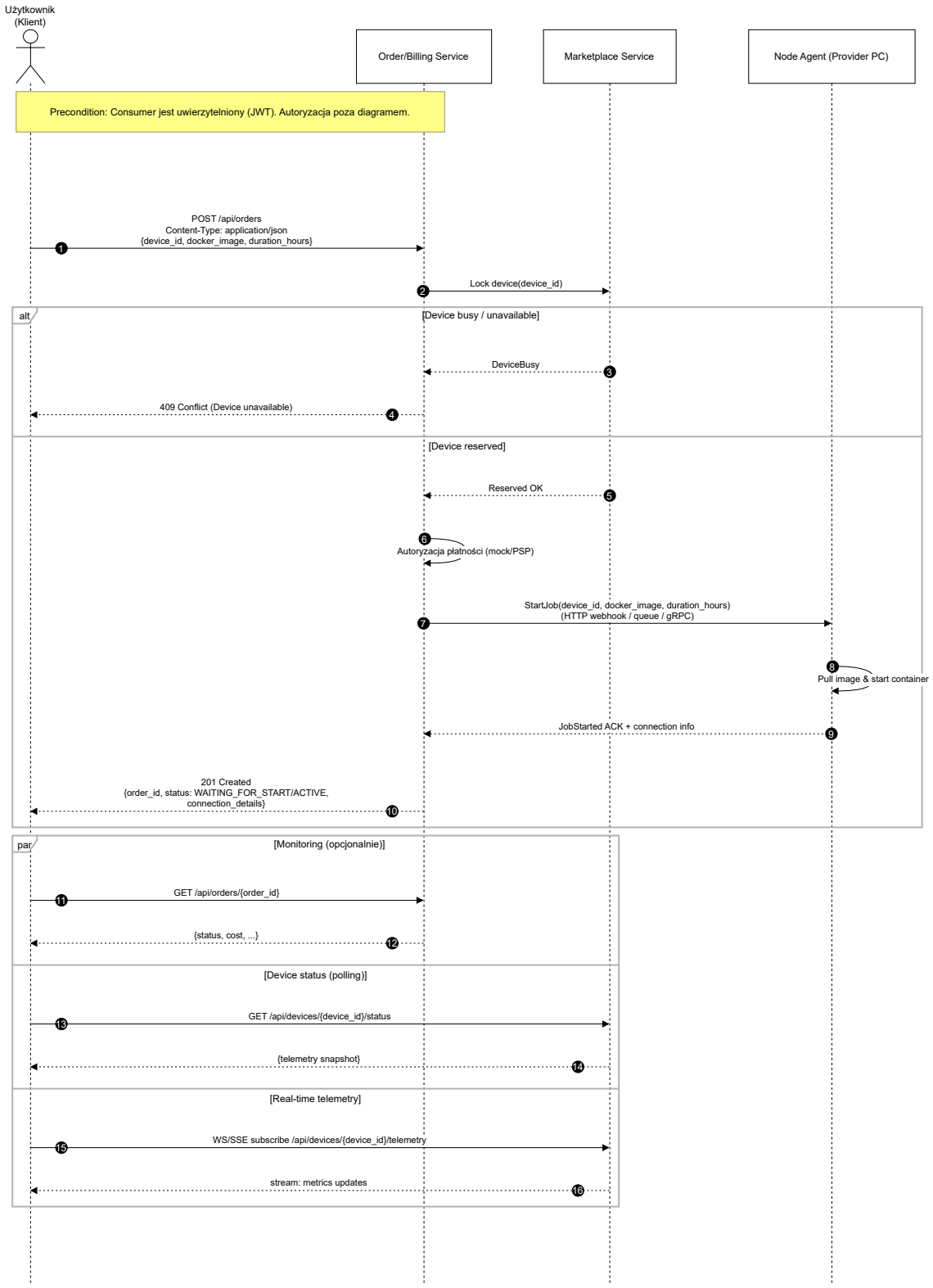
6.4 Sekwencje wymiany wiadomości

6.4.1 Sekwencja: Rejestracja GPU



Rysunek 2: UML Use Cases Diagram

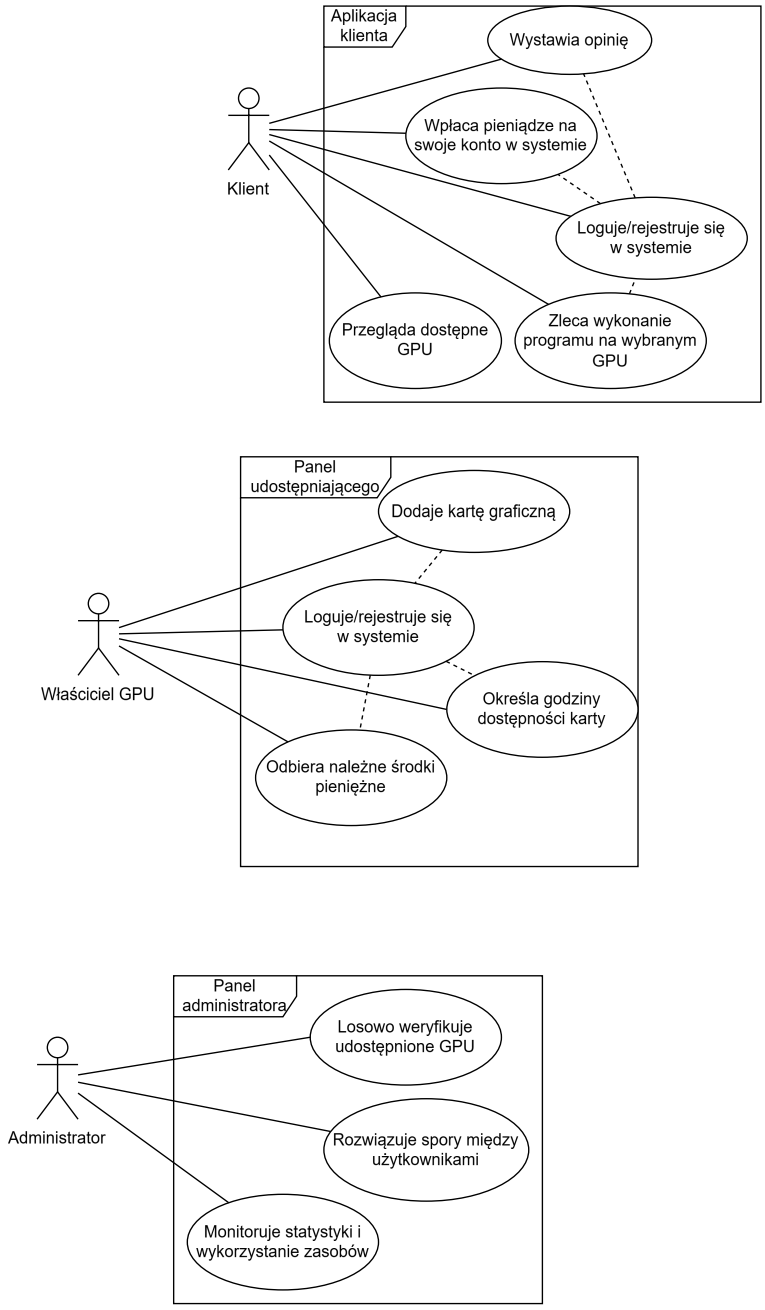
6.4.2 Sekwencja: Wynajem GPU



Rysunek 3: UML Use Cases Diagram

7 Przypadki użycia / historyjki i kryteria akceptacji

7.1 Diagram przypadków użycia



Rysunek 4: UML Use Cases Diagram

7.2 Historyjki użytkowników

Jako...	**Chcę...**	**Aby...**	**MoSCoW**
Użytkownik (klient)	wyszukiwać dostępne karty graficzne według parametrów (np. model, VRAM, cena, lokalizacja)	szybko znaleźć GPU najlepiej dopasowane do moich potrzeb	Must
	móc wypożyczyć wybraną kartę graficzną na określony czas	korzystać z mocy obliczeniowej bez konieczności zakupu sprzętu	Must
	śledzić status aktywnego wypożyczenia (czas pozostały, wydajność, koszty)	mieć kontrolę nad wynajętym zasobem	Should
	móc zgłosić ewentualne niezgodności fizycznego GPU z ogłoszeniem	uniknąć nieprawidłowego obciążenia pieniężnego	Should
	wystawić opinię po zakończeniu wypożyczenia	pomóc innym użytkownikom w wyborze zaufanych właścicieli GPU	Could
	personalizować wygląd interfejsu (motywy, kolory)	dostosować aplikację do własnych preferencji wizualnych	Won't
Właściciel GPU	dodawać swoje karty graficzne do systemu z opisem parametrów i ceną	udostępniać sprzęt innym użytkownikom	Must
	monitorować stan i obciążenie udostępnionej karty	upewnić się, że sprzęt jest prawidłowo używany	Should
	otrzymywać automatyczne płatności po zakończeniu wypożyczenia	mieć gwarancję sprawiedliwego rozliczenia	Must
	oferować rabaty lojalnościowe użytkownikom	zachęcić do częstszego korzystania z usług	Won't
Administrator	weryfikować konta użytkowników i właścicieli GPU	zapewnić bezpieczeństwo i wiarygodność platformy	Must
	rozwiązywać spory między użytkownikami (np. w przypadku nadużyć lub awarii)	utrzymać wysoki poziom zaufania w systemie	Should
	monitorować ogólne statystyki i wykorzystanie zasobów	usprawniać działanie platformy i planować rozwój	Could
	tworzyć automatyczne rekomendacje cen dla właścicieli	zoptymalizować stawki rynkowe	Won't

Tabela 3: Tabela user stories dla systemu GPUShare

7.3 Rejestracja GPU

Aktor: Właściciel GPU
Warunek wstępny: Właściciel jest zalogowany (JWT).
Scenariusz główny:

1. Właściciel uruchamia Node Agent na maszynie z GPU.
2. Node Agent wysyła żądanie rejestracji `POST /api/devices`.
3. System waliduje dane i zapisuje urządzenie.
4. System ustawia status urządzenia jako `AVAILABLE` lub `UNAVAILABLE` (w zależności od ustawień dostępności).

Kryteria akceptacji:

- Urządzenie otrzymuje unikalne `device_id`.
- Urządzenie jest widoczne w katalogu (Marketplace).
- Niepoprawne dane są odrzucane z kodem 4xx oraz opisem błędu.

7.4 Wynajem GPU

Aktor: Klient
Warunek wstępny: Klient ma środki / metodę płatności.
Scenariusz główny:

1. Klient wybiera urządzenie w katalogu.
2. Klient składa zamówienie `POST /api/orders`.
3. System rezerwuje GPU i uruchamia środowisko obliczeniowe przez Node Agent.
4. Klient otrzymuje status oraz dane połączenia i monitoruje przebieg.

Kryteria akceptacji:

- Zamówienie przechodzi do stanu aktywnego lub oczekującego na start zgodnie z logiką systemu.
- Klient może pobrać status wypożyczenia przez `GET /api/orders/{id}`.

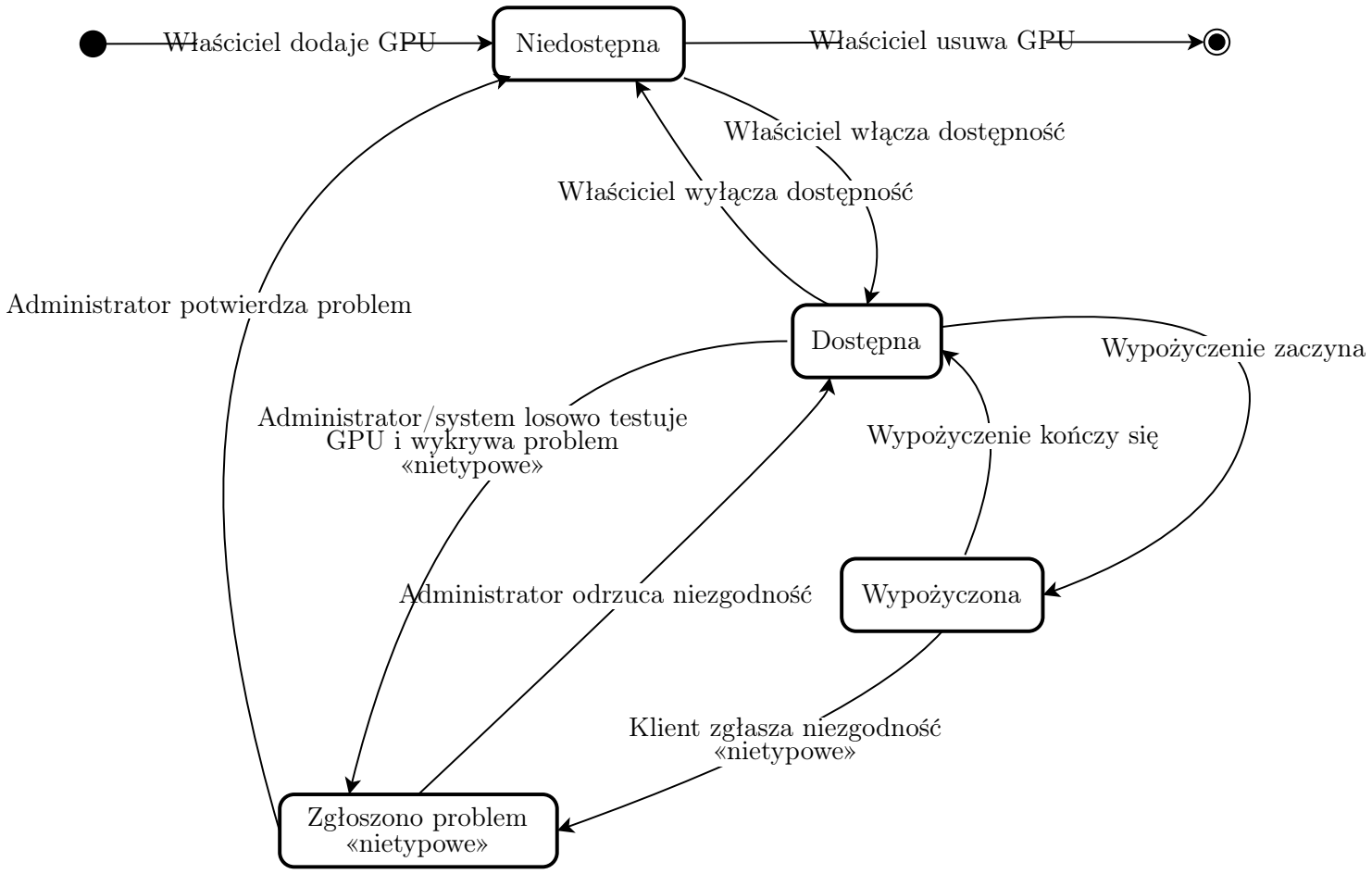
7.5 Zgłoszenie sporu

Aktor: Klient
Scenariusz główny:

1. Klient zgłasza spór `POST /api/disputes`.
2. System ustawia stan sporu na `NEW` i przypisuje go do administratora.
3. Administrator rozstrzyga spór, co kończy proces.

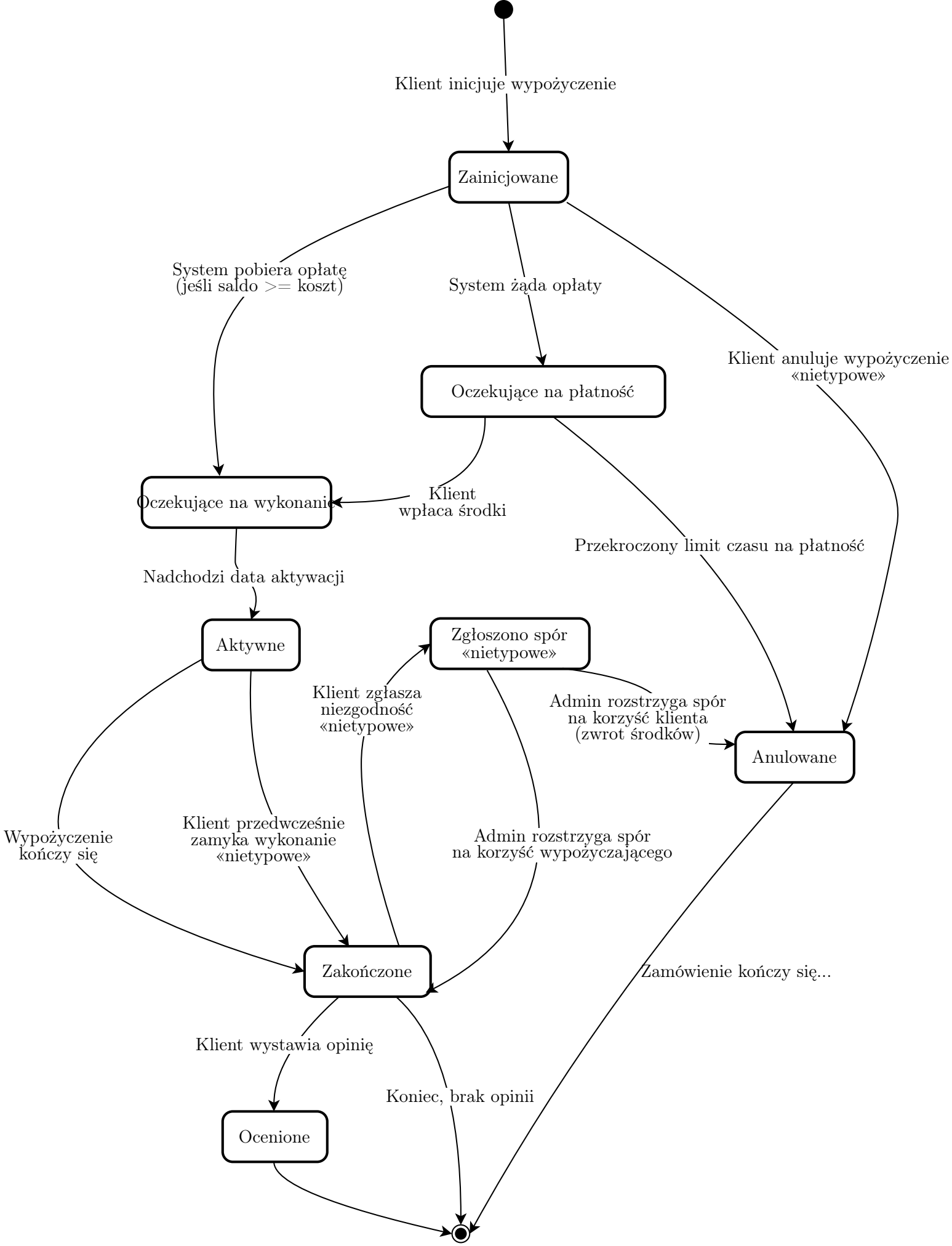
8 Diagramy stanów

8.1 Diagram stanów GPU



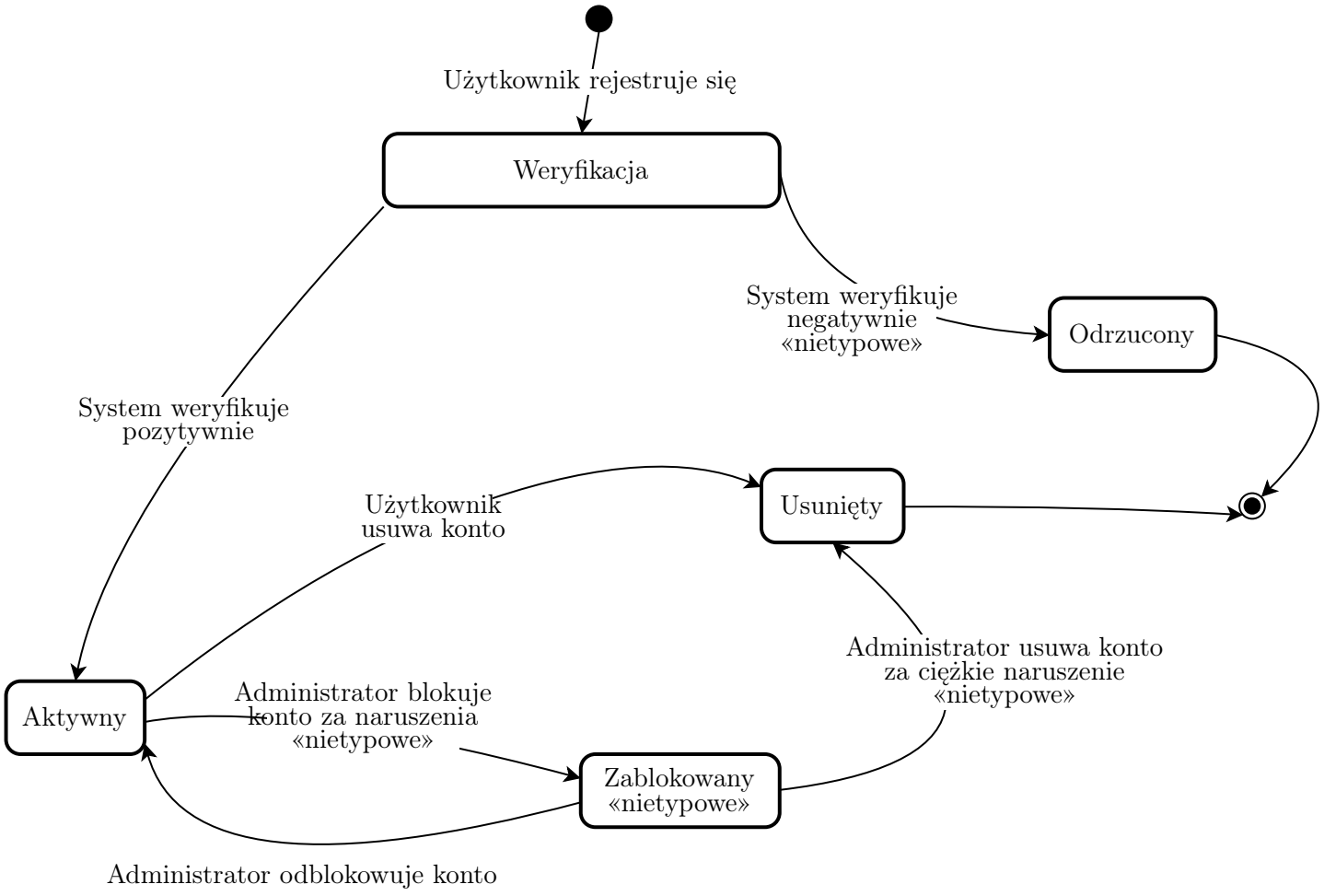
Rysunek 5: Diagram stanów GPu

8.2 Diagram stanów wypożyczenia



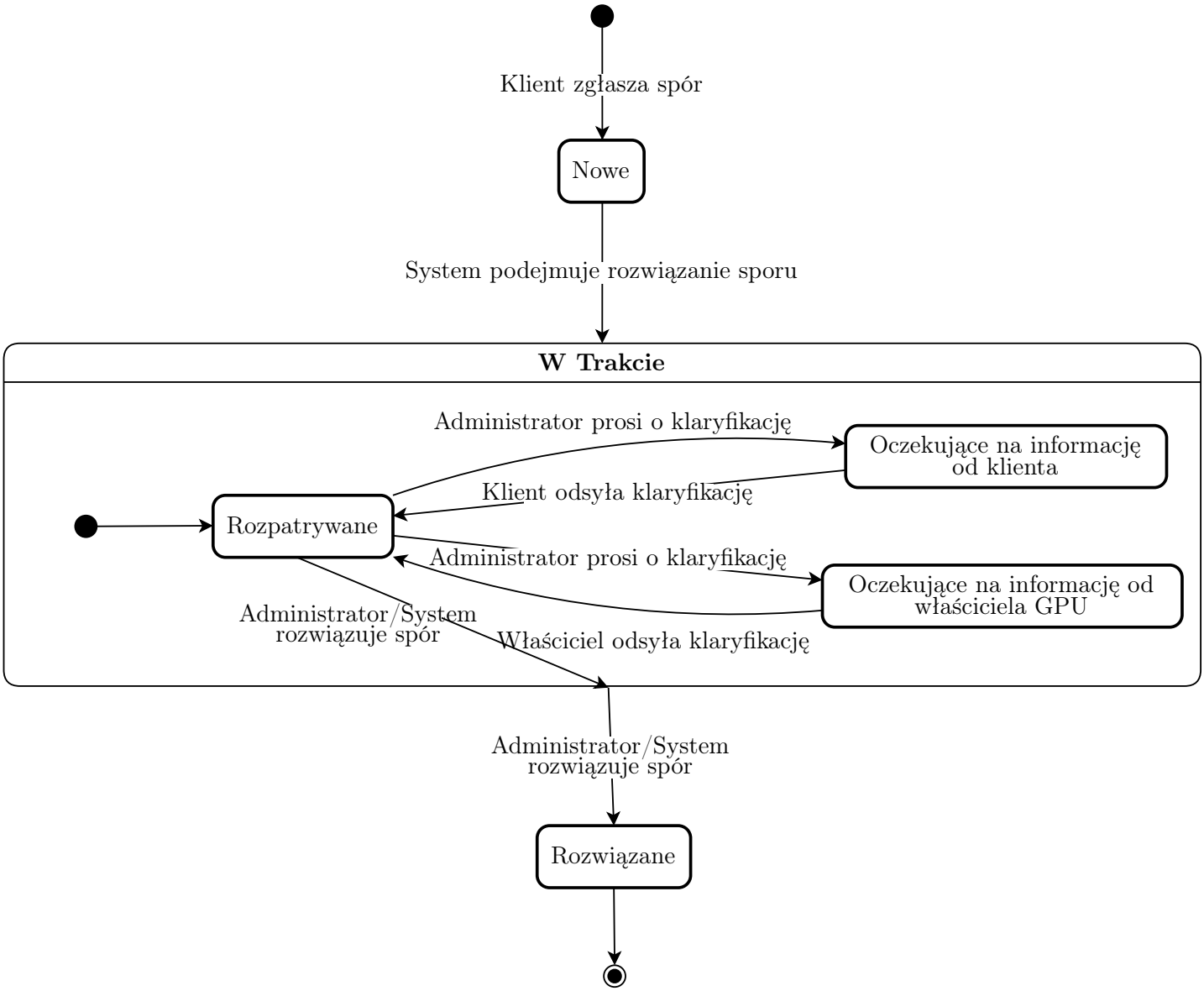
Rysunek 6: Diagram stanów wypożyczenia

8.3 Diagram stanów użytkownika



Rysunek 7: Diagram stanów użytkownika

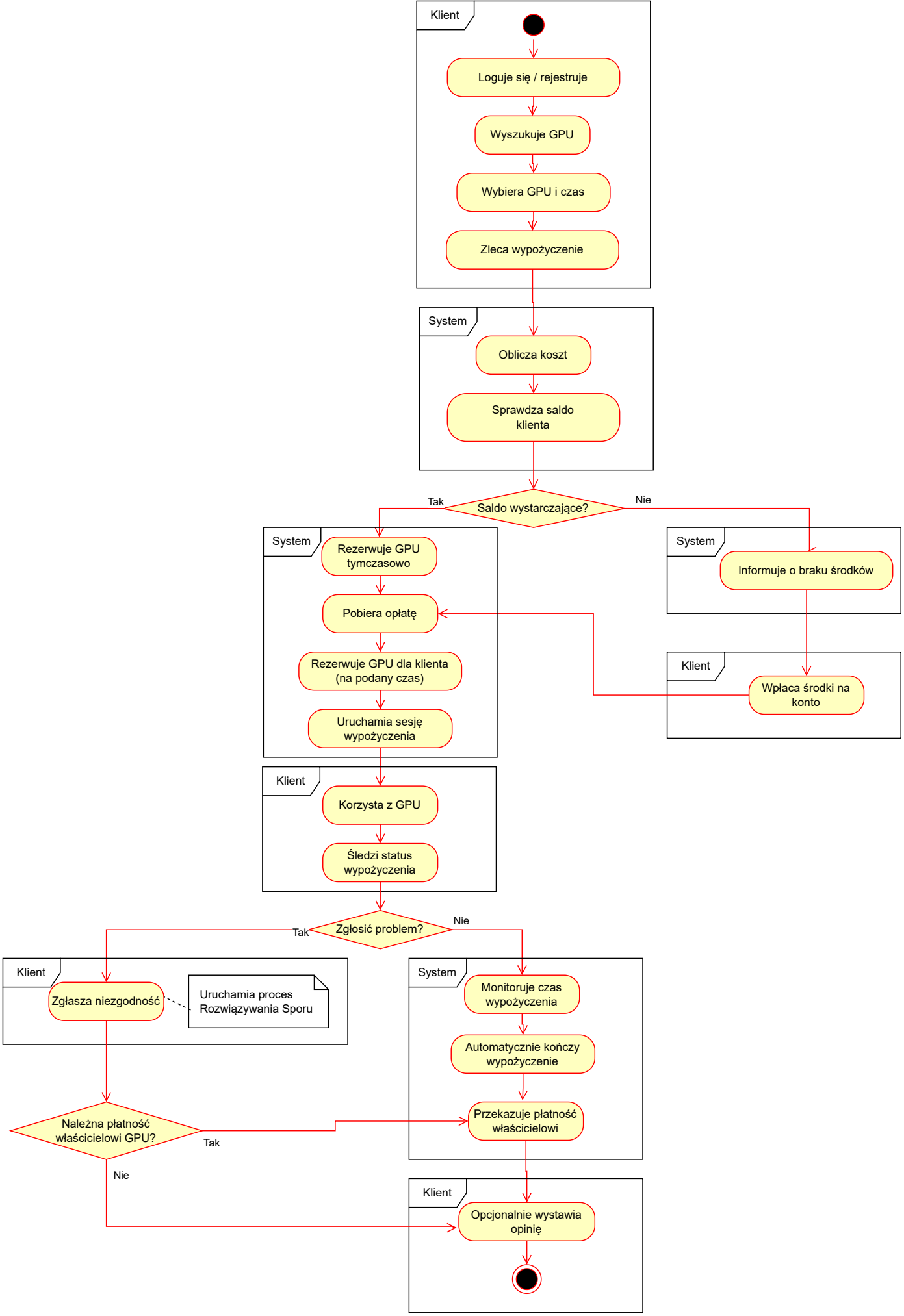
8.4 Diagram stanów sporu



Rysunek 8: Diagram stanów sporu

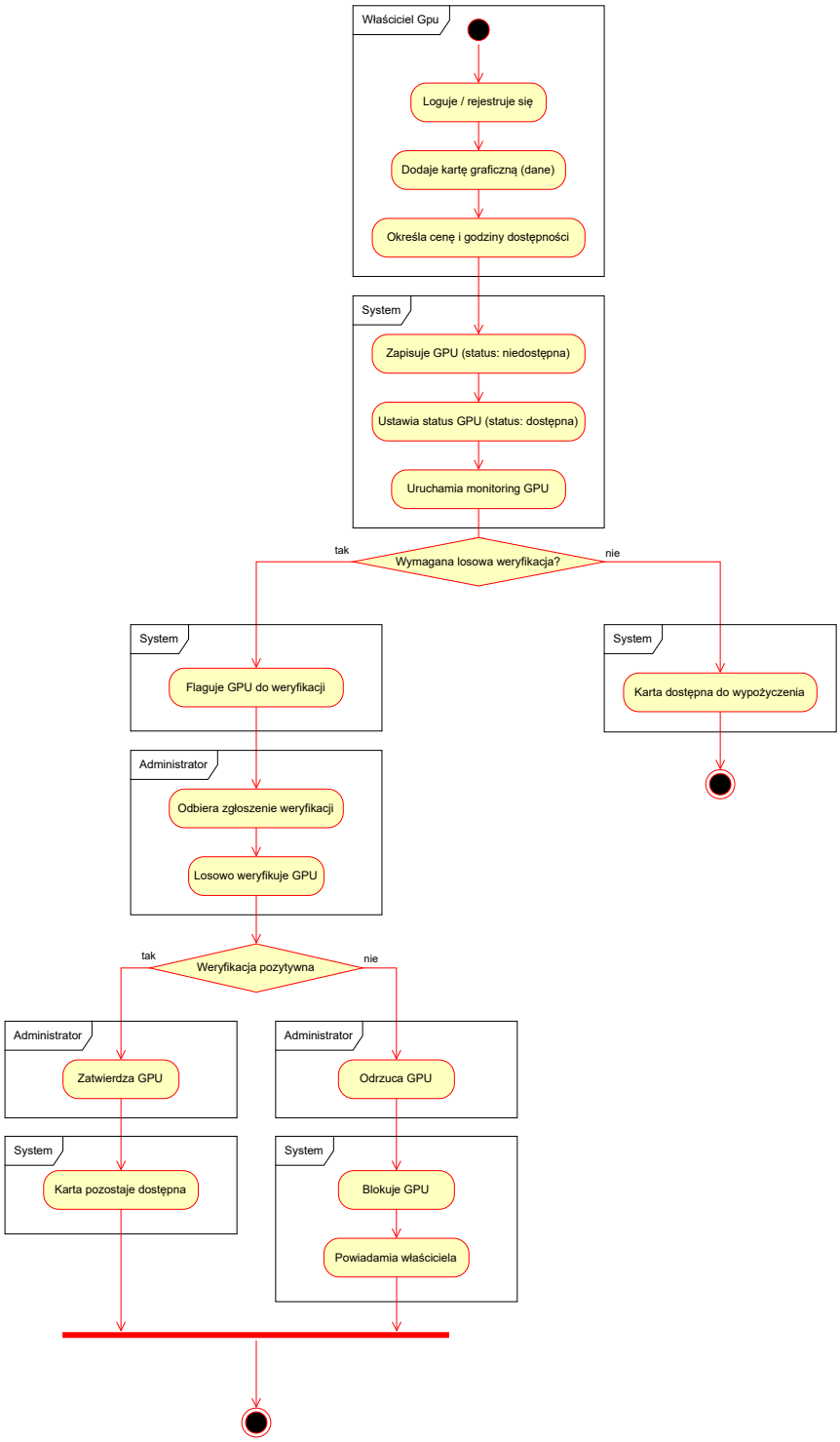
9 Diagramy aktywności

9.1 Diagram aktywności wypożyczenia



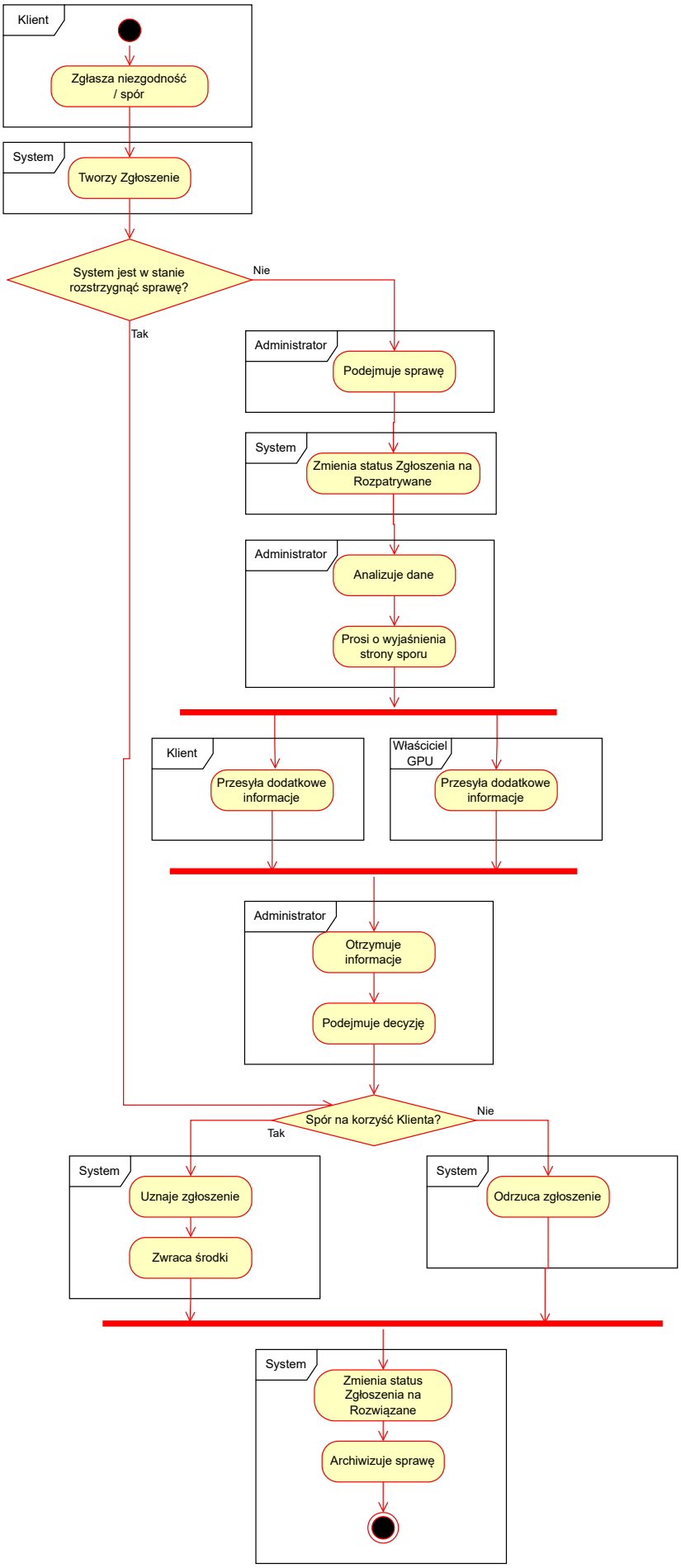
Rysunek 9: Diagram aktywności wypożyczenia

9.2 Diagram aktywności udostępniania GPU



Rysunek 10: Diagram aktywności udostępniania GPU

9.3 Diagram aktywności sporu



Rysunek 11: Diagram aktywności sporu